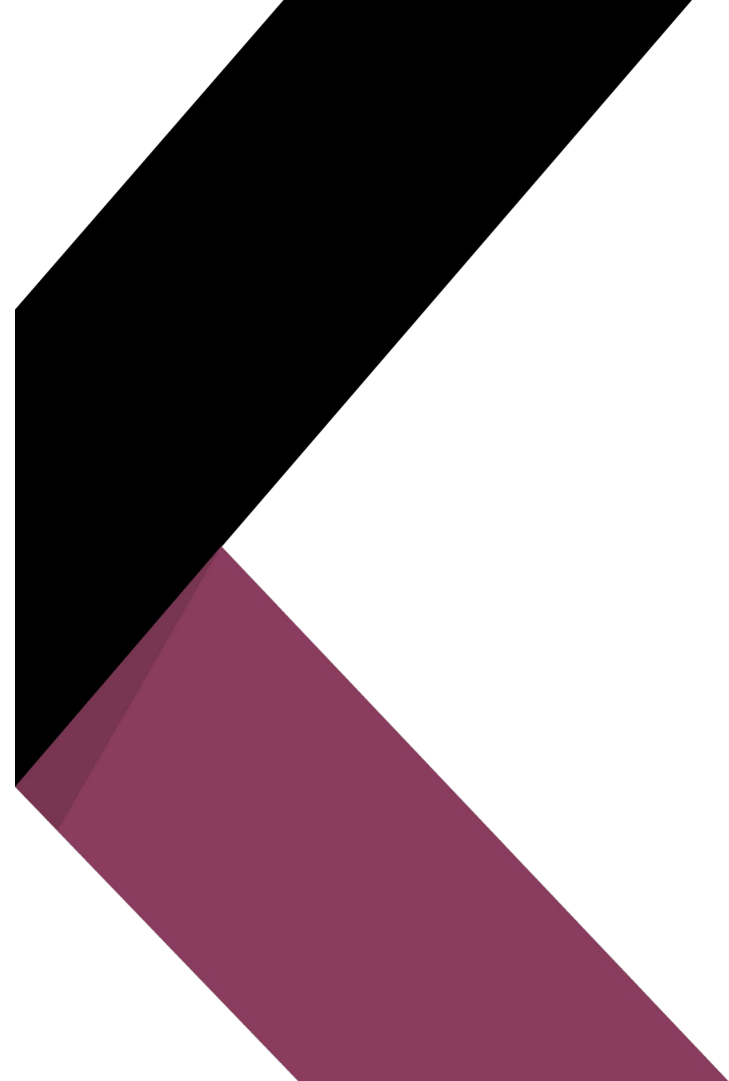


Test Automation

vTestStudio



Overview

- Introduction
- Create new Project
- Test Table Editor
- Test Sequence Diagram
- State Diagram
- Parameters & Variants handling



Introduction



Overview

- vTESTstudio is a powerful development environment for creating automated ECU tests;
- It provides:
 - programming-based,
 - table-based,
 - graphical test notations and test development methods.



Test Table Editor

Easy definition of test sequences
by pre-defined test commands

Easy readability by summary of
command parametrization

Comfortable support of test step
parameterization by drag & drop

The screenshot displays the Test Table Editor interface for a file named 'LockingSystemTester.vit'. It is divided into several panes:

- Test Execution Tree:** A hierarchical view of the test sequence. It includes a 'Test Case' named 'Test velocity dependent requirements of the door control unit'. Under this, there are sub-items like 'Test static requirements of the door control unit', 'Test velocity dependent requirements of the door control unit', 'Lock by increasing velocity', 'Apply crash with different velocities', 'Crash detection when car is moving', 'Lock dependent on velocity and crash detection', 'Lock with car moving/not moving', 'Comfort close', and 'Diagnostics'.
- Functions:** A list of reusable functions and test cases. It includes 'Test Sequence Definitions', 'Test Case Definitions', 'AmbiguousKeyRequest', 'LockingBehavior', 'VelocityDependentLocking', and 'Function Definitions' (with 'ResetSignals' listed).
- Command List:** A table of commands used in the test sequence. The commands are: 'Test Fixture', 'Fixture Preparation', 'Test Case', 'Preparation', 'Set', 'AccelerateDecelerateToTargetSpeed', 'Wait', 'Check', 'AccelerateDecelerateToTargetSpeed', 'Wait', 'Check', 'AccelerateDecelerateToTargetSpeed', 'Completion', 'Comment', 'Test Sequence', 'Test Case', 'Test Case [list]: LockingBehavior', 'Comfort close', and 'Test Case'. Each command has a corresponding 'Caption' or 'Description'.
- Value Table:** A table used for parameterizing test steps. It shows a 'Check' step with a value of 'LockState' and a 'Caption' of 'RecentlyUsed'. A dropdown menu is open, showing a list of symbols: 'CrashDetected', 'CrashDetected', 'ErrorCrashSensorOnVelocity', and 'ErrorCrashSensorUsage'.

Definition of reusable
test cases and functions

Direct calls to CAPL and C#
test cases and functions possible

Access to
value table
entries
defined in
databases

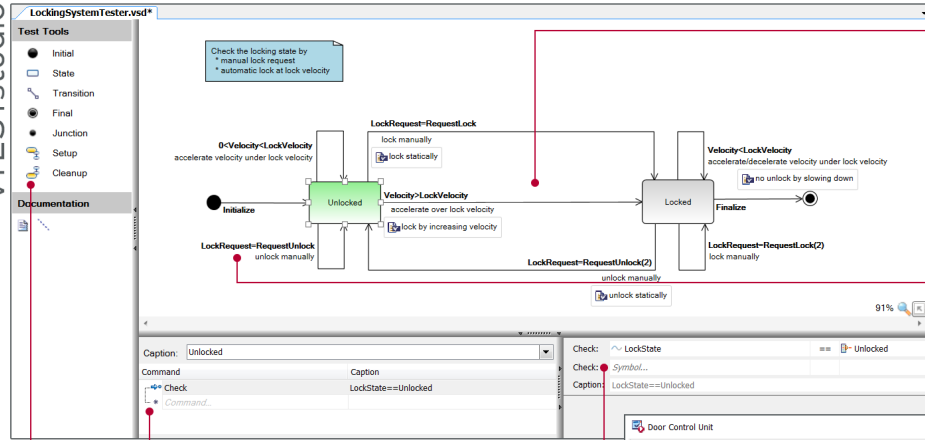
Autocomplete for symbols





State Diagram

vTESTstudio



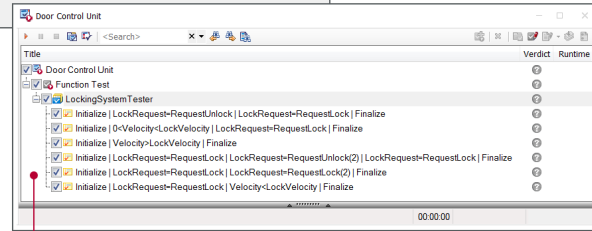
Expected behavior of the SUT modeled as state transition diagram

Extension of the system model to test model e.g. by (self-)transitions as unspecified scenarios

Access to symbols by drag & drop and autocomplete

Usage of pre-defined commands for:
- stimulation and checks
- user-defined functions, e.g. in CAPL C#

Definition of state diagrams by drag & drop of graphical elements out of a toolbox



Automatic test case generation based on transition coverage; several generation algorithms supported

CANoe



Programming Editor

Definition of event
handlers for signal
value pre-processing

Definition of interface functions for
usage in other test design languages

```
112 // Simulates increasing/decreasing velocity slowly as if controlled by driver.
113
114 export testfunction AccelerateDecelerateToTargetSpeed(float targetSpeed)
115 {
116     float currentSpeed;
117
118     TestStep("Speed Change", "Requested speed is %f", targetSpeed);
119     currentSpeed = $VehicleMotion::Velocity;
120     if (targetSpeed > currentSpeed)
121     {
122         // Accelerate
123         TestStep("Speed Change", "Accelerating...");
124         @sysvar::SystemUnderTest::Accelerate = 1;
125         @sysvar::SystemUnderTest::
126     {
127         do
128         {
129             testWaitForTimeout(100);
130             currentSpeed = $VehicleMotion::Velocity;
131             TestStep("Speed Change", "Decelerating...");
132         } while (targetSpeed > currentSpeed);
133         @sysvar::SystemUnderTest::Accelerate = 0;
134     }
135     else if (targetSpeed < currentSpeed)
136     {
137         // decelerate
138         TestStep("Speed Change", "Decelerating...");
139         @sysvar::SystemUnderTest::Accelerate = 0;
140         @sysvar::SystemUnderTest::Decelerate = 1;
141     }
142 }
```

The screenshot shows the Kineton Programming Editor interface. On the left is a file explorer (navigator) with a tree view showing folders like 'Includes', 'Variables', 'System', 'Value Objects', 'Diagnostics', 'Ethernet', 'LIN', 'Service Signal', 'TCP/IP', 'XCP/CCP', 'Functions', 'Test Functions', and 'Test Cases'. The main area is a code editor displaying a C++-like test function. A callout menu is open over the line starting with '@sysvar::SystemUnderTest::', showing a list of symbols to autocomplete: 'Accelerate', 'Decelerate', 'ErrorCrashSensorOnVelocity', 'ErrorCrashSensorUsage', 'ErrorInWindowApplication', and 'ErrorKeyUpAndDownError'. The 'Accelerate' option is highlighted. The code editor also features syntax highlighting and error underlining.

Autocomplete with
support of symbols
as well

Test-specific API

Navigator for easy
navigation within one file

Error underlining

Syntax highlighting

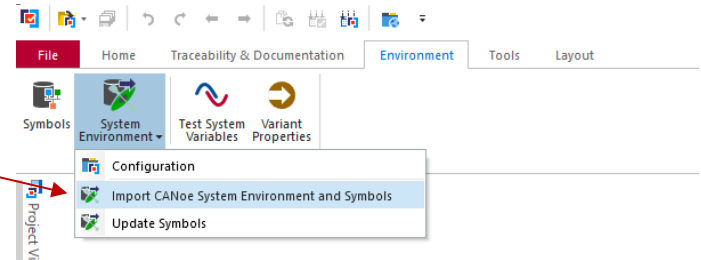
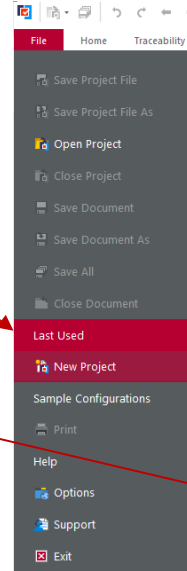


Create new Project



Creating a Project

- From file menu select **New** and select the path where you want to save your project. Give it a name.
- In order to have same naming and variables, Open CANoe and load your configuration, then Import this configuration inside vTestStudio
- Done that, you are ready to start working on your test cases.



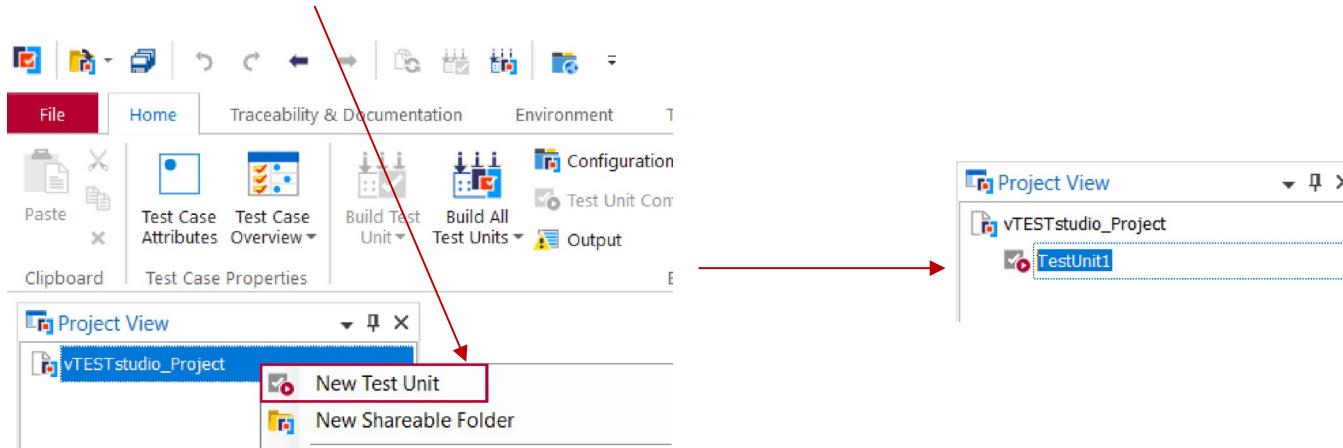


Test Table Editor



Create a Test Unit

- 0 Open the **Project View** (**Layout | Views | Project View**) if not already open.
- 0 Right-click on the created project
- 0 Select **New Test Unit**

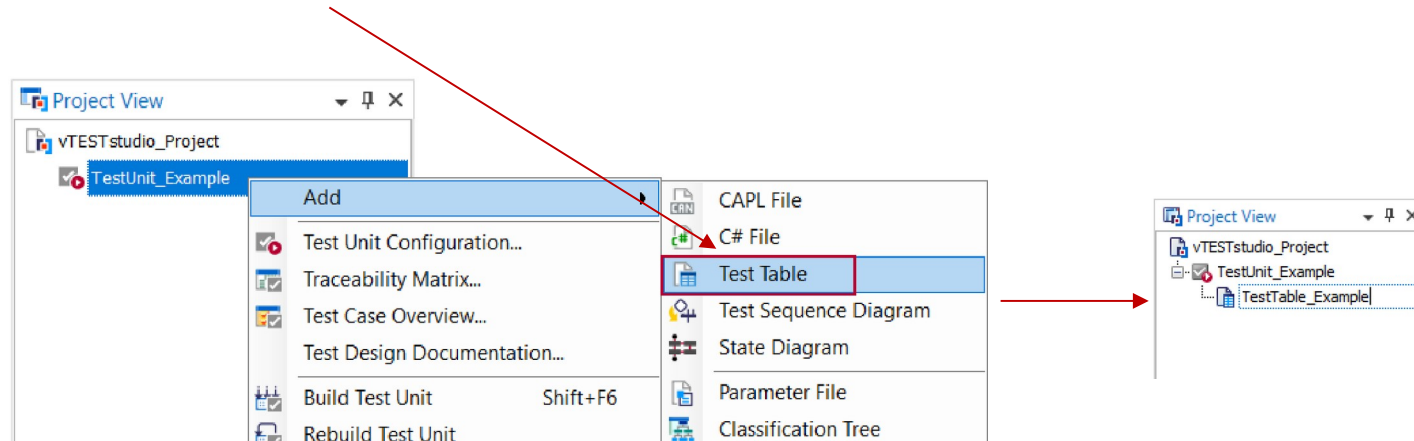




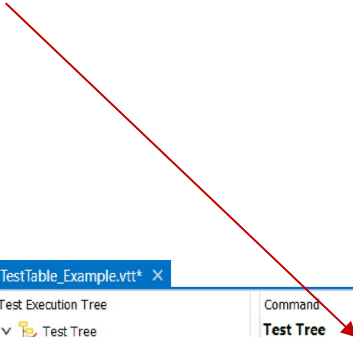
Create a

Test Table

- 0 Right-click on the created Test Unit
- 0 Select **Add | Test Table** from the context menu of the created project



- ~~input assistance will open.~~

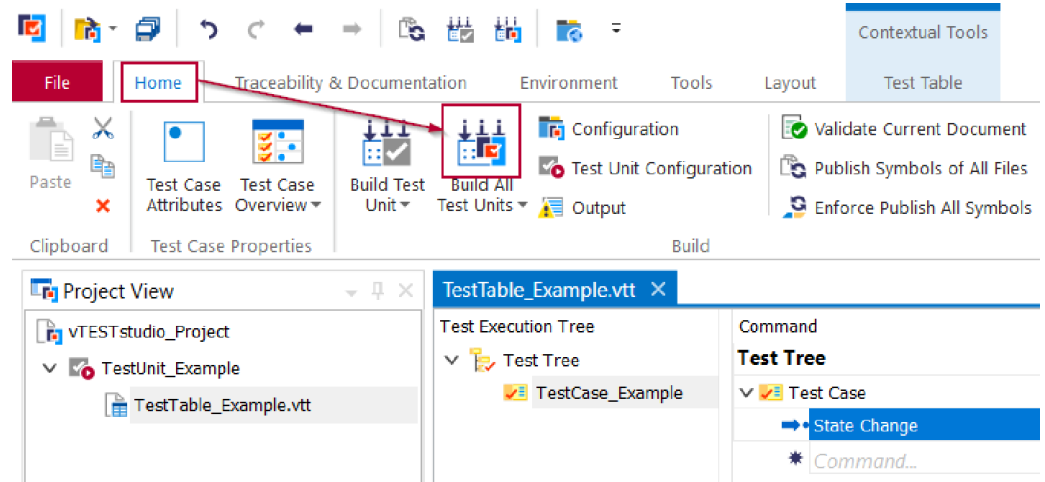




Build a

Test Unit 1/2

- 0 Test Units have to be compiled, before they can be used in CANoe.
- 0 In the vTESTstudio select **Home | Build All Test Units** from the context menu
- 0 The **Output** window shows the results of the built Test Unit. Errors or warnings are displayed here if the build was not successful.



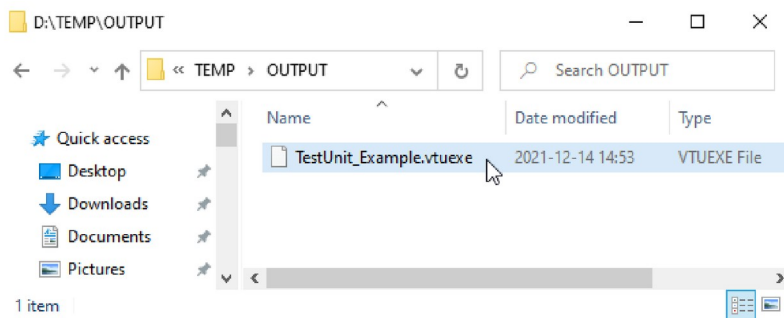
Output		
Description	File	Test Unit
• Checking test unit consistency		TestUnit_Example
• Compiling file 'TestTable_Example.vtt' in test unit 'TestUnit_Example'	TestTable_Example.vtt	TestUnit_Example
• Compilation succeeded (11:32:22) -- 0 errors, 0 warnings	TestTable_Example.vtt	TestUnit_Example
• Test unit created 'D:\SupportNote\vTESTstudio_Project\TestUnit_Example\TestUnit_Example.vtuexe'		TestUnit_Example
Build succeeded (11:32:22) -- 0 errors, 0 warnings		TestUnit_Example
All test units were built successfully.		



Build a

Test Unit 2/2

- 0 An executable Test Unit with file format VTUEXE is now generated in the folder which was defined in **Project|Configuration... | Build**

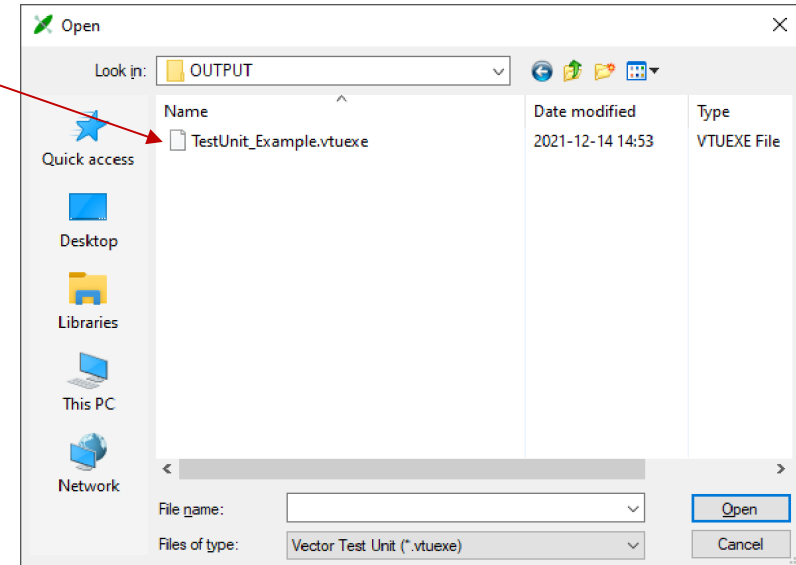
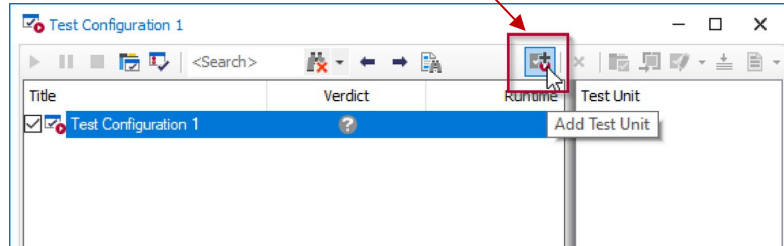




Configure a

Test Unit in CANoe 2/4

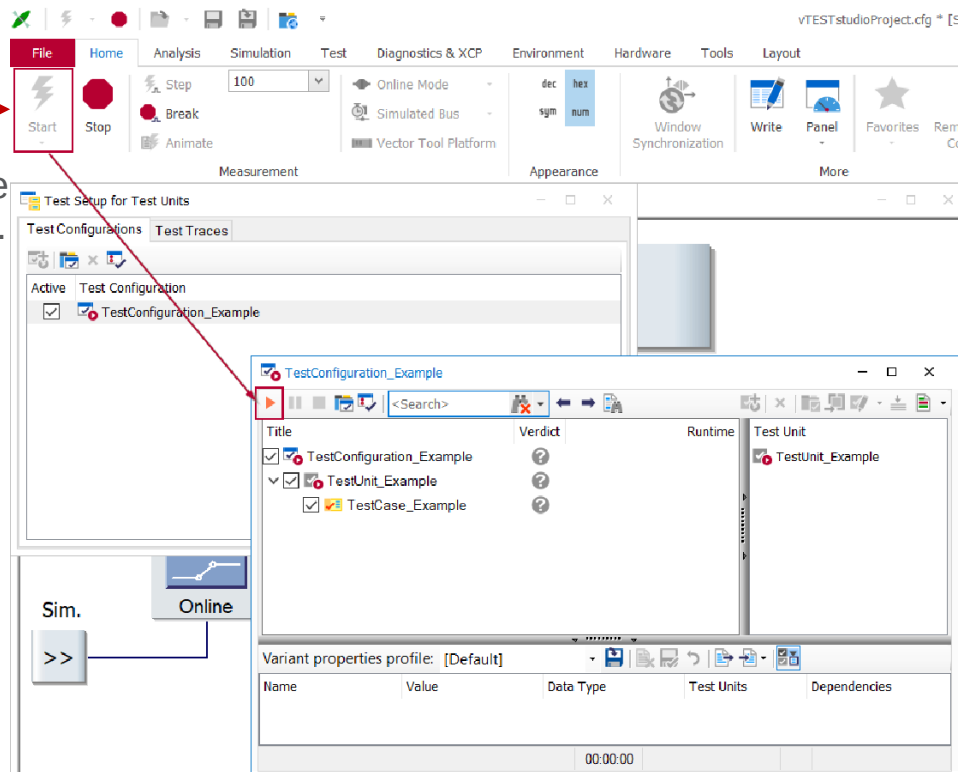
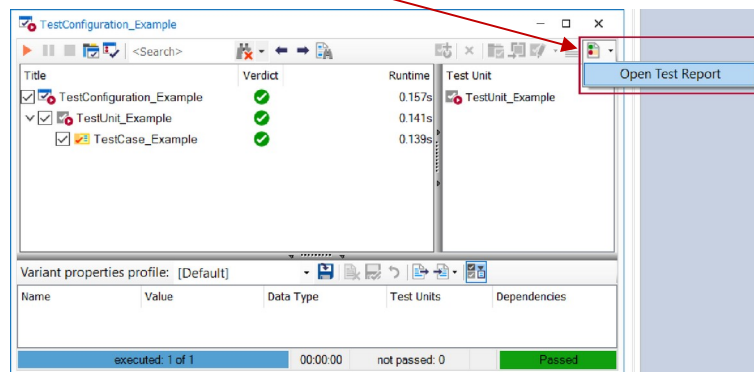
- 0 Select the **Add Test Unit** icon in the test configuration window
- 0 Select the executable test unit according to the output path of the vTESTstudio



Configure a

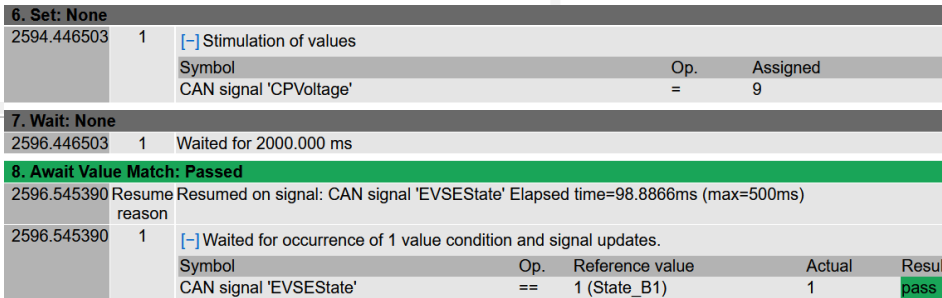
Test Unit in CANoe 3/4

- 0 Start the CANoe measurement [F9].
- 0 Click **Start** Icon to start the test.
The test is executed.
- 0 When the test execution is finished, click the **Report** Icon to view the resulting test report.





0 Test Report





Example

Test Case type

- 0 **Preparation:** Initial conditions
 - 0 **Set:** setting symbol from DBC
 - 0 **Wait:** waiting time for check
 - 0 **Check:** symbol control after set
 - 0 **Completion:** Exit conditions
-
- 0 **Await Value Match:** Check symbol after timeout

Test Case	Test Case
Preparation	Preparation
Set	Set
	Set <symbols>
Wait	1000 ms
	Wait for 1000 ms
Check	Check
	Check <symbols>
Completion	Completion

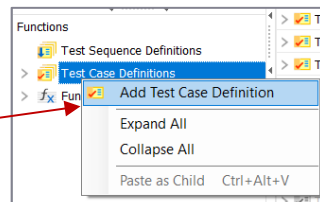
Test Case	Test Case
Preparation	Preparation
Set	Set
	Set <symbols>
Await Value Match	Await Value Match
	Wait until <symbols> with timeout <timeout>
Command...	
Completion	Completion



Example

Test Sequence 1/2

- 0 **Step 1:** From **Functions** add **Test Case Definition** (is used to manage repetitive tests with different values of signals)
- 0 **Step 2:** Select **Test Case Definition**, add **Name** and **Parameters**
- 0 **Step 3:** Write test with **Symbols = Parameters**



Test Sequence Definitions

Test Case Definitions

Test Case Definition Example

Name:	Example		
Test Case ID:	94595b40-1e57-4138-8e8c-e4e8f27865a0		
Break on Fail:	☐		
Use Property...	Export, Description		
Parameters			
Parameter:	Parameter_value	Int64	
Parameter:	Add Parameter...	Type...	
Variables			
Variable:	Variable_value	Float	
Variable:	Variable...	Type...	Value...
Use Property...	Info, Trace Items, Precondition, External Refer ...		

Test Case Definitions	
Test Case Definition	Example
Set	BMS_SOC=Parameter_value
Set	BMS_1_STATUS_1::BMS_SOC=Parameter_value
Command...	

Set:	~ BMS_1_STATUS_1::BMS_SOC	=	Parameter_value
Set:	Symbol...	=	
Caption: BMS_SOC=Parameter_value			



Example

Test Sequence 2/2

- 0 **Step 4: Test Tree | Command |** Type **Test Sequence** and then test case definition **Name**
- 0 **Step 5: Add Parameter Value**

The screenshot displays a test sequence tree on the left and a properties panel on the right. The tree has a root node 'Test Sequence' with a subnode 'Test Case: Example'. The 'Test Case: Example' node is selected, and its properties are shown in the panel on the right. The properties include a signature, a parameter value, a test case ID, a status, an author, and an author last modified date. There are also links to 'Use Property...' for 'Properties of Definition, Description' and 'Info, Trace Items, Variant Dependencies, ...'. The caption is 'Example (Parameter_value=)'.

Test Sequence	Test Sequence
Test Case: Example	Example (Parameter_value=)
	Call Example (Parameter_value=)
* Command...	

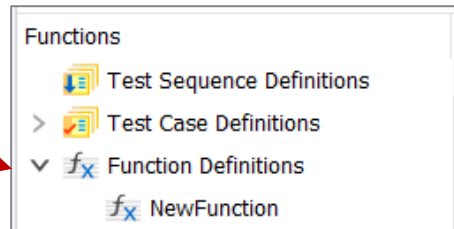
Signature:	Example(Int64 Parameter_value) : void
Parameter_value:	
Test Case ID:	7a26f20a-49f4-4e85-981f-9ef4305a4832
Status:	DRAFT
Author:	
Author last modified:	
Use Property...	Properties of Definition, Description
Use Property...	Info, Trace Items, Variant Dependencies, ...
Caption:	Example (Parameter_value=)



Example

Function

- 0 **Function** is used to manage repetitive pieces of test, to avoid rewriting the same commands
- 0 From **Functions** add **Function Definition** and insert command
- 0 Call function from test case





System Variables 1/2

- Open CANoe
- Click **Environment | System Variables**
- Add new Variable according to DBC

The screenshot displays the CANoe software interface with the following components visible:

- Main Window:** Shows the 'Environment' tab selected in the top menu bar. The left sidebar contains a tree view of system variables, including 'CONSTANTS', 'CAN', 'DIAG', 'SYS', 'Vb_Charge_Complete', 'VCU_HW', 'INPUT', 'OUTPUT', 'FAN_COOLER_ENABLE', 'HVAC_ENABLE', 'K15_WAKE_RELAY_OUT', 'LED_BLU', and 'PWM MeasTime wait s'.
- System Variable Configuration Dialog:** Opened from the 'Environment' tab. It shows a list of 'User-Defined' variables under the 'CONSTANTS' namespace, including 'CAN_OPEN', 'BMS_X', 'DCDC', 'HMI', 'OBC', 'VCU', 'BATTERY_ID_ROUTINE', 'DCDC_COMMAND', 'DIAGNOSTIC_RSP_2_TO_HMI', 'OBC_COMMAND', 'VB_CLUSTER_CMD', 'VCU_CHARGE_INFO', 'VCU_INFO_1', 'VCU_CUR_STATE', 'Charge', and 'Drive'.
- New System Variable Dialog:** Opened from the 'System Variable Configuration' dialog. It shows the configuration for a new variable named 'CAN_OPEN' in the 'CONSTANTS' namespace. The 'Data type' is set to 'Integer (32 Bit signed)' and the 'Location' is 'C:\...\Canoe_cfg\CustomFile\SysVar\CONSTANTS.xml'.

The 'New System Variable' dialog includes the following fields and options:

- Namespace:** CONSTANTS
- Name:** CAN_OPEN
- Comment:** (empty)
- Data type:** Integer (32 Bit signed)
- Location:** C:\...\Canoe_cfg\CustomFile\SysVar\CONSTANTS.xml
- Properties:**
 - Values:**

Initial Value	Minimum	Maximum	Unit
 - Value Table:** None
 - Access and Events:**
 - Only Used in Analysis Mode: ☐
 - Read Only: ☐
 - Multi-CANoe Write Access: ☐
 - Events on Value Assignment: One Event with Last Value



System Variables 2/2

0 Variable association to test

The screenshot displays a software interface with a table of test case steps and a 'Symbols' panel on the right.

Test Case	Preparation
Set	VCU_CUR_STATE=Drive
Set	VCU_INFO_1::VCU_CUR_STATE=VCU_STATUS_1::VCU_CUR_STATE::Drive

Set:		
VCU_INFO_1::VCU_CUR_STATE	=	VCU_STATUS_1::VCU_CUR_STATE::Drive
Symbol...	=	

The 'Symbols' panel on the right shows a search bar and a list of environment variables:

- Environment variables
- System variables

A red arrow points from the text 'Variable association to test' to the variable assignment 'VCU_INFO_1::VCU_CUR_STATE = VCU_STATUS_1::VCU_CUR_STATE::Drive' in the table.

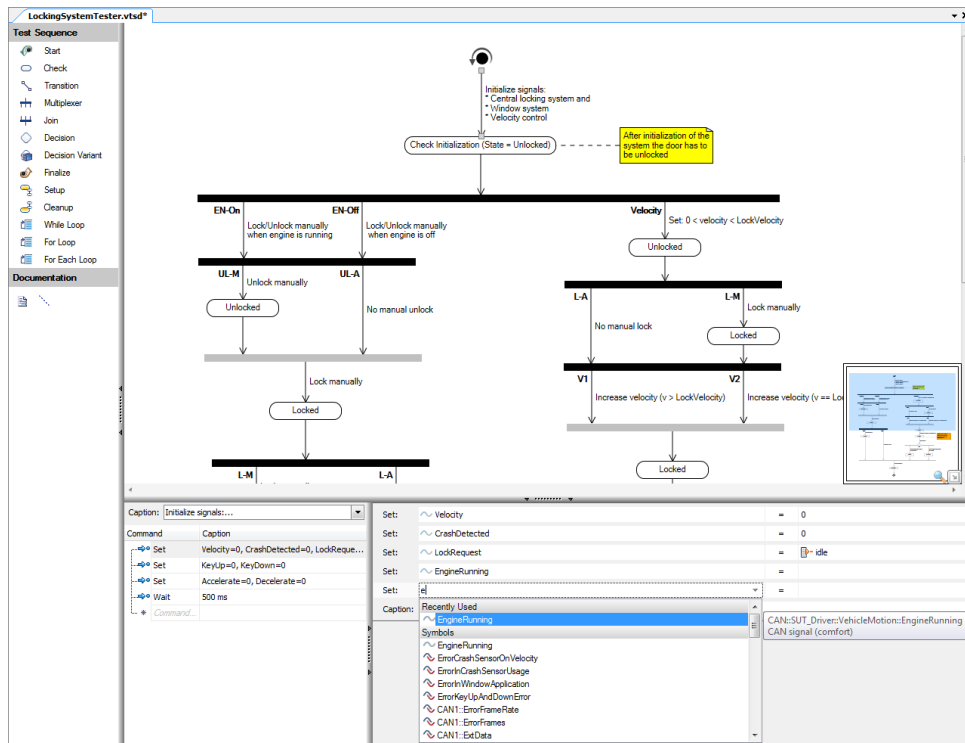


Test Sequence Diagram



Overview Test Sequence Diagram

- The Test Diagram Editor can be used to define tests in a graphical way. The graphical notation is particularly suitable for reviews. Test code in tabular notation is located behind each graphical element. By default one test case is generated for each path through the diagram; a more fine-grained test case definition is possible as well.





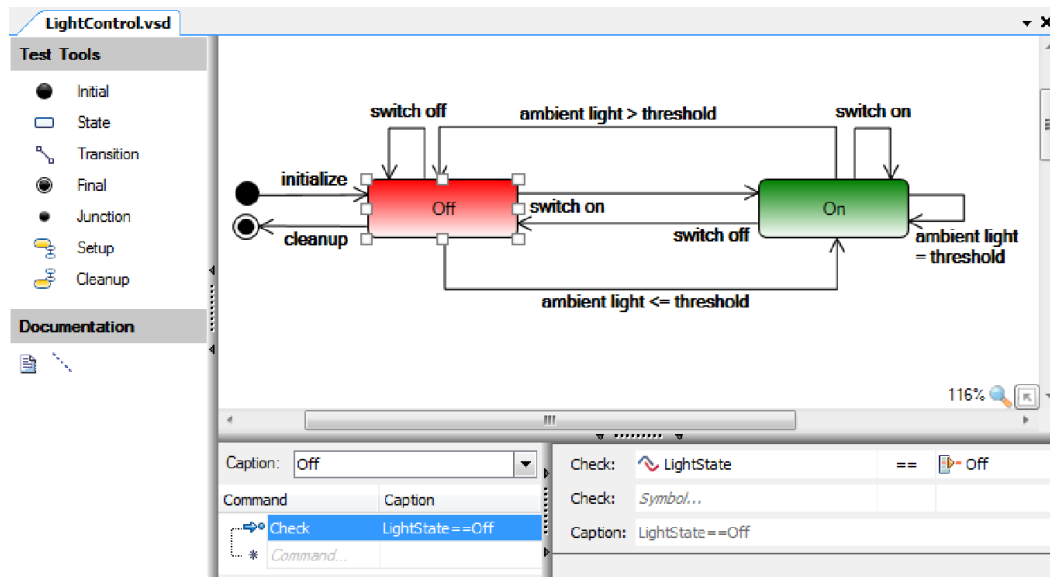
State Diagram



Overview

State Diagram

- The State Diagram Editor can be used to model the expected behavior of the system under test as state model. Test code in tabular notation is located behind each graphical element. Based on transition coverage test cases are automatically generated out of the model. Different generation algorithms are supported, e.g. Chinese Postman algorithm and a breadth search based algorithm.





Parameters & Variants Handling



Parameters

- 0 The term "parameter" refers to any constant value that can be accessed within the test sequence from all implementation languages
- 0 Available parameters are displayed in the Symbol Explorer within the tab **Parameters**.

There are several kinds of parameters:

- 0 **(Scalar) Parameter:** A scalar parameter represents exactly one constant value that can be accessed in the test sequence.
- 0 **(Scalar) List Parameter:** A list parameter has 1...n values for the same variable. In the test sequence, iteration over all values of the list can be performed in order, for example, to perform a test under different temperature values.
- 0 **Struct Parameter:** A struct parameter represents a set of associated (scalar) values. In the test sequence, the individual values of a struct can be accessed in order, for example, to apply a test vector (stimulating and expected values) to the test system.
- 0 **Struct List Parameter:** A struct list parameter corresponds to a list of value tuples for a struct. In the test sequence, iteration over all list elements can be performed in order, for example, to apply various definitions of a test vector onto the system.

Name	Value
CycleTimeTolerance	50

Name	Value
OutsideTemperature	-40, -10, 0, 15, 30, 50

Struct	Member	Value
LockingTestVector	Velocity	60
	CrashDetected	0
	Wait	500
	LockState	1

Struct	Member / Value			
LockingTestVectorList	Velocity	CrashDetected	Wait	LockState
	60	0	500	1
	40	0	500	0
	60	1	250	0



Variants

- 0 ECU variants and test variants can be realized using so-called variant properties.
- 0 Variant properties can be used for conditional test coding as well as for access to variant-dependent parameter values and for defining variant-dependent test cases or test groups.
- 0 Variant properties can be easily transferred to the test code from the Symbol Explorer or using text completion.

Variant Property	Possible Values
Region	US, Europe, Asia
Coverage	full, low, medium
Model	OEM1, OEM2

CONTACTS

Roberto Sorrentino

roberto.sorrentino@kineton.it

Roberta Adinolfi

roberta.adinolfi@kineton.it

Via E. Gianturco 23,
Complesso Mecfond
80146 Napoli

www.kineton.it





Test Unit in CANoe 1/4

Configure a

- Open the CANoe configuration from which the system environment was imported to vTESTstudio.
- Select **Test | Test Setup | Test Configurations**.
A new window **Test Configurations for Test Units** opens now.
- Click **Add Test Configuration** icon.

